Block-Diffusion OD Planning — Experimental Results

Status (2026-07-14). mask blk4 and graph blk4 (d = 2.0) are final. graph blk4 (d = 0.05) is queued for retraining: its first run's checkpoint was truncated by a server disk-full incident, and a recurring GPU-0 hardware fault (second same-day failure, both under dual-GPU load; Xid-79 class) currently blocks new CUDA processes until a reboot. The d-sweep comparison and respaced sampling for the graph kernel will be appended after recovery. Companion report: `RESULTS.pdf` (full-canvas track).

Setup. BD3-LM-style block diffusion (semi-AR across blocks, discrete diffusion within each block), ported from the prior project's implementation and deliberately pruned: discriminator guidance, the no-<end> graph variant, embedding-only conditioning, and the fixed-step mask sampler were removed. Transformer denoiser (6 layers, dim 256, adaLN conditioning, 6.83M params), block size 4, trained 1 epoch = 48.2k iters (bs 32, lr 5e-4) on `porto_v3_normal` (1,390 vertices, 1.93M paths). Two within-block kernels: **mask** — absorbing [MASK] noising, SUBS weighted CE, first-hitting sampling; **graph** — this repo's uniform-state CTMC kernel with the <end>-augmented state space (V+1 states, degree d), D3PM ELBO (KL + CE + connectivity), full 100-step reverse per block.

Length specification (project decision): canvas `[dst, ori, v1, ..., v_L=dst, END...END | PAD...PAD]` — within the block containing the path's final token, every later position is an END token and IS a training target; all subsequent blocks are PAD and are EXCLUDED from the loss. Generation is open-ended: blocks are appended until dst (hit) or END (miss) is emitted — no length model of any kind. Conditioning follows the design validated in the full-canvas track: dst + ori as clean in-context prefix tokens plus the OD embedding through adaLN, condition dropout 0.1, CFG off (w = 1).

Evaluation: 1,000 held-out OD pairs, open-ended (`length_mode=open`), variants raw / refine_nx (shortest-path patch to dst, guarantees arrival) / refine_es (`eval_shortest.refine()`). hit = reaches dst unaided (= raw arrival); **valid∧hit** = usable-path rate; len corr as in the companion report.

---

1. Architecture and mechanics

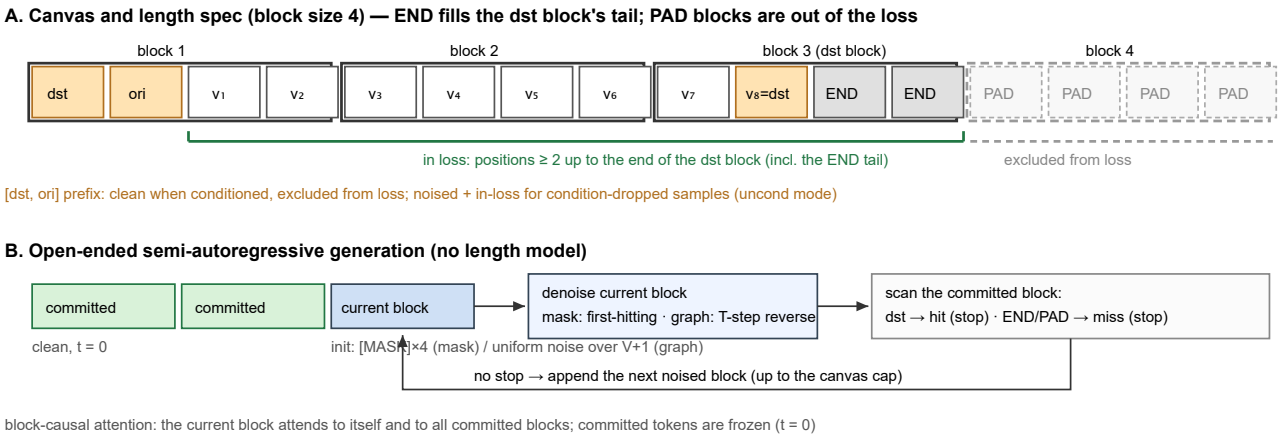

Figure 1. Block-diffusion mechanics. **A**: the canvas per the project length spec — the destination is an in-context token at position 0 (plus the OD embedding channel), the END tail is confined to (and trained within) the block that contains the path's final token, and every later block is PAD, excluded from the loss. **B**: generation appends one noised block at a time; the freshly denoised block is scanned and generation stops at dst (hit) or END (miss) — length emerges from the model, with no length module. The denoiser network itself is shown in Figure 2.

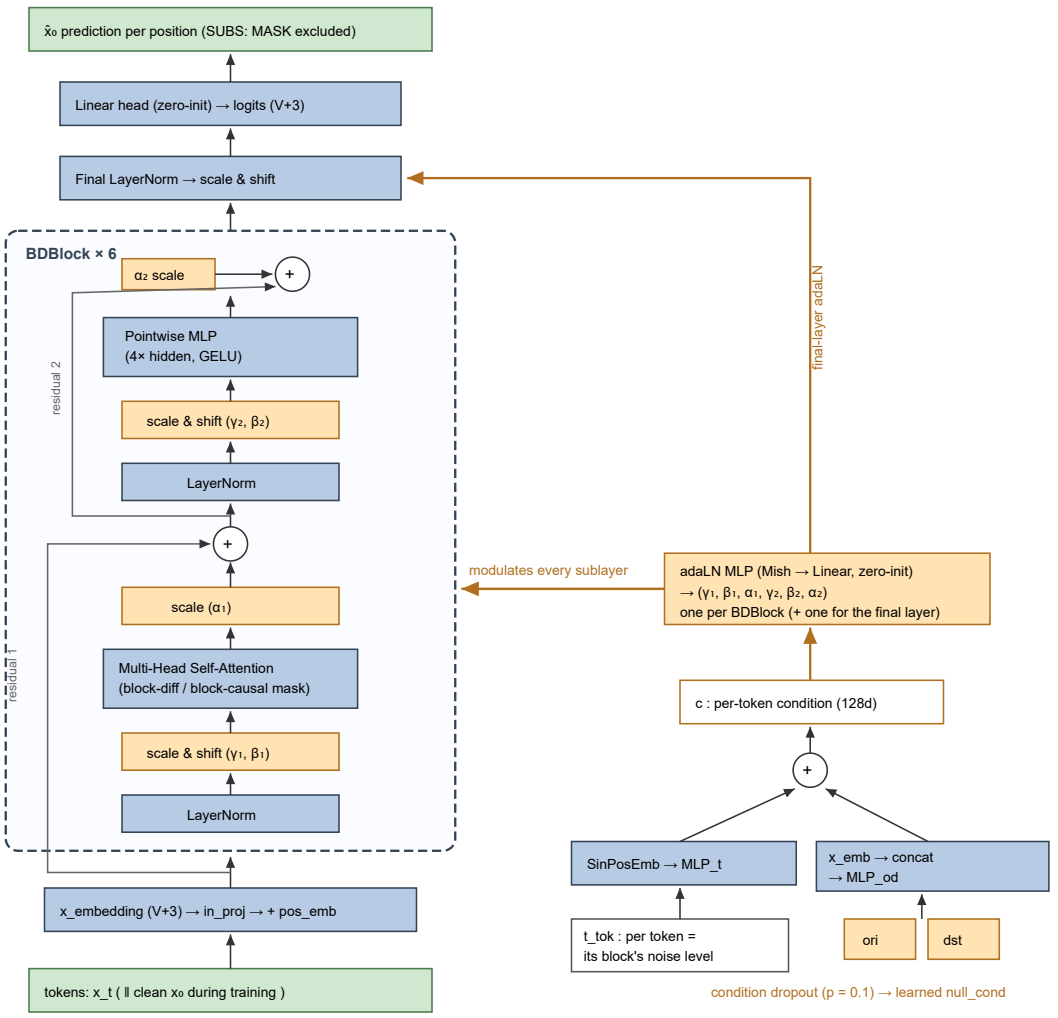

Figure 2. The denoiser is a standard **DiT tower** (Peebles & Xie style), read bottom-up: token embeddings plus learned positions enter 6 pre-LN transformer blocks whose LayerNorms are *affine-free* — all scales and shifts come from the condition. For each block, a dedicated zero-initialised adaLN MLP maps the per-token condition `c = MLP_t(t) + MLP_od([e(ori); e(dst)])` to six modulation vectors: shift/scale before each sublayer (γ, β) and a gate after it (α); zero-init makes every block start as the identity, and the final layer receives its own scale & shift. Because t is per *token* and each block of the canvas carries its own noise level, one forward pass handles blocks at different corruption stages — the property that enables the concatenated `[x_t ‖ x₀]` training pass (Figure 1B note) and semi-AR sampling with frozen (t = 0) committed blocks. Differences from vanilla DiT: block-structured attention masks instead of full attention, a token head over V+3 states instead of patch reconstruction, and the OD embedding added into c alongside the timestep.

1.1 Where this lives in the code

Canvas + length spec — `models_seq/bd_models.py`, `BlockDiffusion.build_canvas` (asserted in `smoke_test_bd.py`, incl. the dst-on-block-boundary edge case):

```
dst_pos = pfx - 1 + lengths - 1                          # position of the path's final token
end_blk = (dst_pos // block + 1) * block                 # end of the dst block (exclusive)
x0[i, 0] = dst
x0[i, pfx-1 : pfx-1+L] = p                               # [dst, ori, v1, ..., v_{L-1}]
x0[i, pfx-1+L : end_blk[i]] = self.END                   # END tail (in loss; may be empty)
loss_mask = (pos_idx >= pfx) & (pos_idx < end_blk[:, None])  # PAD blocks excluded
```

Vectorized training — one forward over `[x_t ‖ x_0]` with the block-diffusion attention mask:

```
x_input = torch.cat([xt, x0], dim=1)                     # (b, 2n)
attn = get_block_diff_mask(n, block, self.device)        # block-diag | offset-block-causal | block-causal
logits = self.backbone(x_input, t_in, ori, dst, cond_mask, attn, canvas_len=n)[:, :n]
```

adaLN conditioning — `BDBlock.forward` (Figure 2):

```
(shift_msa, scale_msa, gate_msa,
 shift_mlp, scale_mlp, gate_mlp) = self.adaLN_modulation(c).chunk(6, dim=-1)   # zero-init
x = x + gate_msa * self._attention(_modulate(self.norm1(x), shift_msa, scale_msa), attn_mask)
x = x + gate_mlp * self.dropout(self.mlp(_modulate(self.norm2(x), shift_mlp, scale_mlp)))
# c = time_mlp(t_tok) + od_mlp([emb(ori); emb(dst)])  (BDTransformer.forward)
```

Open-ended semi-AR loop + stop rule — `BlockDiffusion.plan`:

```
for j in range(max_blocks):
    seq = torch.cat([seq, self._init_block(b)], dim=1)       # MASK block / uniform-noise block
    if self.kernel == "mask": self._denoise_block_mask(seq, origs, dests, w)
    else:                     self._denoise_block_graph(seq, origs, dests, w, num_mc_samples)
    for tok in committed_block:                              # scan (per sample)
        if tok == dst: stop_idx = ...; break                 # hit  -> truncate at dst
        if tok in (END, PAD, MASK): end_idx = ...; break     # miss -> truncate before END
```

Mask kernel: first-hitting sampler (~1 forward per revealed token) — `_denoise_block_mask`:

```
 t = t * u ** (1.0 / num_masked)                              # first-hitting time update
p_x0 = softmax(logits[:, -block:]);  p_x0[..., MASK] = 0     # SUBS
idx = multinomial(masked_positions)                          # reveal ONE masked position
seq[:, pos[idx]] = multinomial(p_x0[idx])
```

---

2. Kernel comparison: mask vs graph (blk4, d = 2.0)

kernel / variant	hit (pre-refine)	arrival	valid	valid∧hit	EM	PC	DTW	len err	len corr	gen len	s/path
mask, raw	**0.952**	0.952	0.850	**0.820**	0.761	0.797	0.040	**0.38**	**+0.99**	23.4	**0.003**
mask, refine_es	0.952	0.980	0.856	0.825	0.790	0.818	0.038	0.36	—	—	—
mask, refine_nx	0.952	1.000	0.851	0.820	0.753	0.792	0.037	0.48	—	—	—
graph d=2.0, raw	0.099	0.099	**0.971**	0.099	0.320	0.512	0.432	6.89	+0.55	18.6	0.064
graph d=2.0, refine_es	0.099	0.247	0.954	0.099	0.621	0.751	0.432	7.17	—	—	—
graph d=2.0, refine_nx	0.099	1.000	1.000	0.099	0.189	0.375	0.106	4.30	—	—	—

Column semantics: **hit** is always measured on the unrefined model output (hence constant across a kernel's rows), while **arrival** is measured after the row's post-processing. arrival can exceed hit without contradiction because refine_es only requires the path to pass *within one edge* of the destination (it cuts at the first dst-adjacent vertex and appends dst — the path need never contain dst itself), and refine_nx grafts a shortest path to dst, so its arrival is 1.0 by construction. In the raw rows no post-processing exists and hit = arrival.

Arrival by real path length (raw) — the horizon test:

kernel	len 2–15	len 16–25	len 26–35	len 36+
mask	0.906	0.963	**0.975**	**0.959**
graph d=2.0	0.086	0.127	0.088	0.067
full-canvas best (A1+A2+A3, ref.)	*0.132*	*0.080*	*0.046*	*0.017*

- **The mask kernel essentially solves conditional OD planning on this benchmark**: 95.2% of paths reach the destination unaided, 82.0% are simultaneously edge-valid, the emergent length matches the per-pair real length almost perfectly (len corr +0.99, mean 23.4 vs real 23.2, |Δlen| = 0.38), and **arrival does not decay with distance** (0.96 at length 36+) — progressive block commitment eliminates the horizon-compounding failure that capped the full-canvas track at ~8% raw arrival.
- **The graph kernel shows the same validity-vs-arrival trade as in the prior project**: near-perfect validity (0.97, from the adjacency-constrained block decode) but only 9.9% hit — better than the full-canvas graph models raw (7.7%) but a factor ~10 behind the mask kernel, and 23× slower (100-step reverse per block vs ~1 forward per token).
- The new length spec trains cleanly on both kernels (mask masked-CE 0.35 at convergence; graph CE 1.2 / KL 0.005), and END-tail supervision within the dst block is sufficient for stopping — no PAD supervision needed.
- refine_nx guarantees arrival by construction (patch to dst) at a small EM/PC cost; refine_es adds +0.03 arrival for the mask kernel and +0.15 for graph. With raw arrival at 0.95, refinement is nearly obsolete for the mask kernel.

---

3. Relation to the full-canvas track (RESULTS.pdf)

model (raw, w best)	arrival	valid∧arr/hit	len corr	len err
full-canvas base (d=2.0, soft cond)	0.010	0.010	+0.44	7.6
full-canvas + anchored cond (A1+A3)	0.061	0.061	+0.62	6.0
full-canvas + A2 (best full-canvas)	0.077	0.077	+0.47	8.0
BD graph blk4 (d=2.0)	0.099	0.099	+0.55	6.9
BD mask blk4	**0.952**	**0.820**	**+0.99**	**0.38**

- The full-canvas ablations correctly identified the two levers — anchored in-context conditioning and horizon decomposition — and BD inherits both. Keeping the kernel fixed (graph CTMC), moving from full-canvas to blocks buys a modest gain (0.077 → 0.099 raw). **The decisive factor is the within-block kernel**: absorbing/[MASK] noising with first-hitting decoding turns the same transformer, data, and canvas into a 95%-arrival planner.
- Interpretation, consistent with the prior project's report: the mask kernel's $\hat{x}_0$ predictions are sharp (each masked position is either known from context or directly inferable), while the uniform kernel must denoise from a fully shuffled state and its mean-field $\hat{x}_0$ + MC posterior injects noise at every one of 100 steps per block. The full-canvas respacing result (RESULTS.pdf §4.2: fewer steps = better) suggests a large part of the graph kernel's gap is sampling-procedure noise, not representation — respaced block sampling is the queued test.

---

4. Queued work

- **graph blk4 d = 0.05 retrain** (checkpoint lost to the disk-full incident) → the user-requested d = 0.05 vs d = 2.0 length-calibration comparison (len corr / gen len / END behavior) under the block-diffusion length spec.
- **Respaced block sampling** for the graph kernel (composed exact CTMC kernels, `-sample_steps`): the full-canvas sweep found 25 steps strictly better than 100 *and* 3.7× faster; per-block application would multiply the speedup.
- Blocked on: server reboot (GPU-0 fault broke CUDA initialization for new processes; second same-day failure — hardware inspection of GPU 0 and PSU headroom for dual-3090 load recommended; single-GPU sequential scheduling will be used meanwhile).

Artifacts

- Checkpoints: `sets_model/BD_porto_v3_normal_{mask_blk4_base, graph_blk4_d2.0}_bd.pth` (mask verified intact after the incident); results `sets_res/BD_porto_v3_normal_*_bd.res` + per-path EM/PC records under `sets_res/em_pc/`
- Code: `models_seq/bd_models.py` (canvas spec asserted in `smoke_test_bd.py`), `models_seq/bd_trainer.py`, `main_bd.py`, `train_porto_bd.sh`